

mostly asked questions on java 8 in intrevwer with details answer

Here's a list of the **most commonly asked Java 8 interview questions** with **clear and detailed answers** — from basic to advanced — that interviewers often focus on.

1. What are the main features introduced in Java 8?

Answer:

Java 8 introduced several major enhancements:

- **Lambda Expressions** — Enable functional programming and reduce boilerplate code.
 - **Functional Interfaces** — Interfaces with exactly one abstract method (e.g., `Runnable`, `Comparator`, `Function`).
 - **Stream API** — For processing collections in a declarative and functional style.
 - **Default & Static Methods in Interfaces** — Allow adding new methods to interfaces without breaking existing implementations.
 - **Optional Class** — To handle `null` values gracefully.
 - **Date and Time API (`java.time`)** — New API for handling date/time in an immutable and thread-safe manner.
 - **Method References** — Simplify lambda expressions by directly referring to existing methods.
 - **Nashorn JavaScript Engine** — Execute JavaScript code within Java.
 - **Collectors API** — For grouping, partitioning, and reducing stream results.
-

2. What is a Lambda Expression?

Answer:

A **Lambda Expression** is a short block of code that takes parameters and returns a value, allowing functions to be passed as parameters.

Syntax:

java

 Copy code

```
(parameter) -> expression
```

or

```
java
```

 Copy code

```
(parameter) -> { statements }
```

Example:

```
java
```

 Copy code

```
List<String> list = Arrays.asList("A", "B", "C");  
list.forEach(s -> System.out.println(s));
```

Benefits:

- Reduces boilerplate code.
- Enables functional-style programming.
- Works well with the Stream API.

3. What is a Functional Interface?

Answer:

A **Functional Interface** is an interface with exactly one abstract method.

It can have any number of **default** or **static** methods.

Examples:

- `Runnable` (method: `run()`)
- `Callable` (method: `call()`)
- `Comparator<T>` (method: `compare(T o1, T o2)`)

You can mark them using the `@FunctionalInterface` annotation.

Example:

```
java
```

 Copy code

```
@FunctionalInterface
interface MyFunctionalInterface {
    void execute();
}
```

4. What are Method References?

Answer:

Method references provide a way to refer to methods directly without invoking them.

Types of Method References:

1. `Object::instanceMethod`
2. `Class::staticMethod`
3. `Class::instanceMethod`
4. `ClassName::new` (Constructor reference)

Example:

java

 Copy code

```
List<String> names = Arrays.asList("John", "Jane", "Max");
names.forEach(System.out::println); // Instead of names.forEach(s -> System.out.pr
```

5. What is the Stream API?

Answer:

The **Stream API** processes data declaratively (like SQL).

It allows **filtering, mapping, sorting, reducing**, etc., on collections.

Example:

java

 Copy code

```
List<String> names = Arrays.asList("John", "Jane", "Jack", "Tom");
```

```
List<String> filteredNames = names.stream()
    .filter(name -> name.startsWith("J"))
    .sorted()
    .collect(Collectors.toList());

System.out.println(filteredNames); // [Jack, Jane, John]
```

Key Points:

- Streams are **not** data structures.
- They don't modify the original collection.
- They support **lazy evaluation** (operations are executed only when a terminal operation is called).



6. Difference between Stream and Collection?

Feature	Stream	Collection
Storage	Does not store elements	Stores elements
Iteration	Internal iteration	External iteration
Reusability	Can be used only once	Can be reused
Evaluation	Lazy	Eager
Parallelism	Supports easy parallelism	Manual handling needed



7. What is the difference between Intermediate and Terminal operations in Streams?

Intermediate operations: Return another Stream.

Examples: `filter()`, `map()`, `sorted()`, `distinct()`

Terminal operations: Produce a non-stream result (value or collection).

Examples: `collect()`, `forEach()`, `count()`, `reduce()`, `findFirst()`

Example:

java

```
long count = names.stream()
    .filter(s -> s.startsWith("J"))
    .count(); // count() is terminal
```



8. What is Optional in Java 8?

Answer:

`Optional` is a container class used to represent the presence or absence of a value — to avoid `NullPointerException`.

Example:

java

Copy code

```
Optional<String> name = Optional.ofNullable(getName());
System.out.println(name.orElse("Default Name"));
```

Important Methods:

- `isPresent()`
- `ifPresent()`
- `orElse()`
- `orElseGet()`
- `orElseThrow()`



9. What is the new Date and Time API in Java 8?

Answer:

The new `java.time` package replaces the old `Date` and `Calendar` classes.

Key Classes:

- `LocalDate` – Date without time
- `LocalTime` – Time without date
- `LocalDateTime` – Both date and time
- `ZonedDateTime` – Date/time with time zone

- Period and Duration – Represent differences in time

Example:

java

 Copy code

```
LocalDate today = LocalDate.now();
LocalDate nextWeek = today.plusWeeks(1);
System.out.println(nextWeek);
```

10. What are Default and Static Methods in Interfaces?

Answer:

Java 8 allows **default** and **static** methods inside interfaces.

Default method:

Provides a default implementation to avoid breaking existing classes.

java

 Copy code

```
interface Vehicle {
    default void start() {
        System.out.println("Vehicle is starting");
    }
}
```

Static method:

Belongs to the interface itself and cannot be overridden.

java

 Copy code

```
interface Vehicle {
    static void stop() {
        System.out.println("Vehicle stopped");
    }
}
```

⚡ 11. What is the difference between map() and flatMap()?

Method	Description	Example
map()	Transforms each element	Stream.of("java", "spring").map(String::toUpperCase)
flatMap()	Flattens nested streams	Stream.of(list1, list2).flatMap(List::stream)

Example:

java

 Copy code

```
List<List<String>> names = Arrays.asList(
    Arrays.asList("A", "B"),
    Arrays.asList("C", "D")
);
names.stream().flatMap(List::stream).forEach(System.out::println);
// Output: A B C D
```

🔍 12. What is the use of Collectors in Java 8?

Answer:

Collectors provide various reduction operations such as:

- toList()
- toSet()
- joining()
- groupingBy()
- partitioningBy()

Example:

java

 Copy code

```
Map<String, Long> counting = names.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

🌸 13. How to perform parallel processing in Java 8?

Answer:

You can use `parallelStream()` for parallel computation.

Example:

java

📋 Copy code

```
List<Integer> numbers = Arrays.asList(1,2,3,4,5);  
numbers.parallelStream().forEach(System.out::println);
```

Parallel streams internally use the **ForkJoinPool** for splitting and merging tasks.

🧠 14. Difference between Predicate, Function, Supplier, and Consumer?

Interface	Method	Description
Predicate<T>	boolean test(T t)	Tests a condition
Function<T,R>	R apply(T t)	Takes one argument, returns a result
Supplier<T>	T get()	Returns a value, no input
Consumer<T>	void accept(T t)	Takes input, returns nothing

Example:

java

📋 Copy code

```
Predicate<Integer> isEven = x -> x % 2 == 0;  
Function<Integer, Integer> square = x -> x * x;  
Supplier<String> greet = () -> "Hello";  
Consumer<String> print = x -> System.out.println(x);
```

🧩 15. Can you explain Stream's reduce() operation?

Answer:

`reduce()` combines elements of a stream into a single result.

Example:

java

 Copy code

```
int sum = Arrays.asList(1, 2, 3, 4)
    .stream()
    .reduce(0, (a, b) -> a + b);
System.out.println(sum); // 10
```

Would you like me to also share **Java 8 coding questions** (like using Stream API, Optional, or Lambda) that are **commonly asked in interviews**?

These are practical and often come after theory questions.



yes

Perfect 👍 — here's a list of **commonly asked Java 8 coding questions** that interviewers use to test your **practical knowledge** of Lambdas, Streams, Optionals, and Functional Interfaces — along with **detailed explanations and solutions**.



1. Find even numbers from a list using Stream API

Question:

Given a list of integers, print all even numbers.

Example Input:

[1, 2, 3, 4, 5, 6]

Solution:

java

```
import java.util.*;
import java.util.stream.*;

public class EvenNumbers {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

        numbers.stream()
            .filter(n -> n % 2 == 0)
            .forEach(System.out::println);
    }
}
```

Explanation:

- `filter()` keeps only elements satisfying the predicate (`n % 2 == 0`).
- `forEach()` prints each even number.

 2. Find the sum of all numbers in a list**Question:**

Write a Java 8 program to find the sum of integers using streams.

Solution:

java

 Copy code

```
int sum = Arrays.asList(1, 2, 3, 4, 5)
    .stream()
    .mapToInt(Integer::intValue)
    .sum();
```

```
System.out.println(sum); // 15
```

Alternative using reduce():

java

 Copy code

```
int sum = Arrays.asList(1, 2, 3, 4, 5)
    .stream()
    .reduce(0, Integer::sum);
```



3. Convert a list of strings to uppercase

Solution:

java

Copy code

```
List<String> names = Arrays.asList("john", "mary", "alex");

List<String> upperNames = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(upperNames); // [JOHN, MARY, ALEX]
```



4. Remove duplicates from a list

Solution:

java

Copy code

```
List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 4, 4, 5);

List<Integer> unique = numbers.stream()
    .distinct()
    .collect(Collectors.toList());

System.out.println(unique); // [1, 2, 3, 4, 5]
```



5. Count the number of strings starting with a specific letter

Solution:

java

```
List<String> names = Arrays.asList("John", "Jack", "Tom", "Jill");

long count = names.stream()
    .filter(name -> name.startsWith("J"))
    .count();

System.out.println(count); // 3
```

6. Find the maximum and minimum element in a list

Solution:

java

 Copy code

```
List<Integer> numbers = Arrays.asList(10, 20, 5, 40, 15);

int max = numbers.stream().max(Integer::compare).get();
int min = numbers.stream().min(Integer::compare).get();

System.out.println("Max: " + max + ", Min: " + min);
```

7. Sort a list of strings in reverse order

Solution:

java

 Copy code

```
List<String> names = Arrays.asList("John", "Alex", "Bob");

List<String> sorted = names.stream()
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());

System.out.println(sorted); // [John, Bob, Alex]
```

8. Find the first non-repeated character in a string

Question:

Find the first character that appears only once in "swiss" .

Solution:

java

 Copy code

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class FirstNonRepeated {
    public static void main(String[] args) {
        String input = "swiss";

        Character result = input.chars()
                                .mapToObj(c -> (char) c)
                                .collect(Collectors.groupingBy(Function.identity(),
                                                                Collectors.entrySet(),
                                                                Collectors.toList()))
                                .stream()
                                .filter(e -> e.getValue().size() == 1)
                                .map(Map.Entry::getKey)
                                .findFirst()
                                .orElse(null);

        System.out.println(result); // w
    }
}
```

9. Group employees by department

Solution:

java

 Copy code

```
import java.util.*;
import java.util.stream.*;
```

```
class Employee {
    String name;
    String dept;

    Employee(String name, String dept) {
        this.name = name;
        this.dept = dept;
    }

    public String toString() { return name; }
}

public class GroupByExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("John", "IT"),
            new Employee("Alex", "HR"),
            new Employee("Sam", "IT")
        );

        Map<String, List<Employee>> byDept = employees.stream()
                                                        .collect(Collectors.groupingBy(e -> e.dept));

        System.out.println(byDept);
    }
}
```

Output:

 Copy code

```
{HR=[Alex], IT=[John, Sam]}
```

10. Use Optional to handle null safely

Question:

You have a method `getUserName()` that may return `null`.

Write a safe way to print the name or "Unknown" .

Solution:

java

 Copy code

```
String name = null;

String result = Optional.ofNullable(name)
    .orElse("Unknown");

System.out.println(result); // Unknown
```

11. Get the second highest number from a list

Solution:

java

 Copy code

```
List<Integer> numbers = Arrays.asList(1, 3, 4, 2, 5);

int secondHighest = numbers.stream()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(-1);

System.out.println(secondHighest); // 4
```

12. Find frequency of each element in a list

Solution:

java

 Copy code

```
List<String> names = Arrays.asList("John", "Alex", "John", "Sam");

Map<String, Long> freq = names.stream()
```

```
.collect(Collectors.groupingBy(Function.identity(),
```

```
System.out.println(freq);
```

```
// Output: {Alex=1, John=2, Sam=1}
```

13. Join all strings with a comma

Solution:

java

 Copy code

```
List<String> names = Arrays.asList("Java", "Spring", "AWS");
```

```
String result = names.stream()  
                    .collect(Collectors.joining(", "));
```

```
System.out.println(result); // Java, Spring, AWS
```

14. Calculate the average of a list

Solution:

java

 Copy code

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);
```

```
double average = numbers.stream()  
                        .mapToInt(Integer::intValue)  
                        .average()  
                        .orElse(0.0);
```

```
System.out.println(average); // 25.0
```

15. Flatten a list of lists

Solution:

java

 Copy code

```
List<List<String>> nestedList = Arrays.asList(
    Arrays.asList("A", "B"),
    Arrays.asList("C", "D")
);

List<String> flatList = nestedList.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());

System.out.println(flatList); // [A, B, C, D]
```

Would you like me to send a **separate set of advanced Java 8 coding questions** (like